```python
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime


def create_employee_file():

    data = """Rahul,45000
Sneha,52000
Amit,61000
Priya,47000
Kiran,39000"""

    with open("/tmp/employees.txt", "w") as file:
        file.write(data)

    print("Employee file created successfully")


def read_employee_data():

    with open("/tmp/employees.txt", "r") as file:

        lines = file.readlines()

        print("Reading Employee Records")

        for line in lines:
            print(line.strip())


def calculate_salary_expense():

    total_salary = 0

    with open("/tmp/employees.txt", "r") as file:

        lines = file.readlines()

        for line in lines:

            name, salary = line.strip().split(",")

            total_salary += int(salary)

    print(f"Total Salary Expense = {total_salary}")

    return total_salary
```

```
def find_highest_salary():

    highest_salary = 0
    employee_name = ""

    with open("/tmp/employees.txt", "r") as file:

        lines = file.readlines()

        for line in lines:

            name, salary = line.strip().split(",")

            salary = int(salary)

            if salary > highest_salary:
                highest_salary = salary
                employee_name = name

    print(f"Highest Salary = {highest_salary}")
    print(f"Employee = {employee_name}")


def generate_salary_report():

    report = """Employee Salary Report

Total Employees = 5

Total Salary Expense = 244000

Status = Processed Successfully"""

    with open("/tmp/salary_report.txt", "w") as file:
        file.write(report)

    print("Salary report generated successfully")


with DAG(

    dag_id="employee_salary_processing",

    start_date=datetime(2026, 1, 1),

    schedule=None,
```

```
    catchup=False,

    tags=["employee", "salary", "practice"]

) as dag:

    task1 = PythonOperator(
        task_id="create_employee_file",
        python_callable=create_employee_file
    )

    task2 = PythonOperator(
        task_id="read_employee_data",
        python_callable=read_employee_data
    )

    task3 = PythonOperator(
        task_id="calculate_salary_expense",
        python_callable=calculate_salary_expense
    )

    task4 = PythonOperator(
        task_id="find_highest_salary",
        python_callable=find_highest_salary
    )

    task5 = PythonOperator(
        task_id="generate_salary_report",
        python_callable=generate_salary_report
    )

    task1 >> task2 >> task3 >> task4 >> task5
```

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime



def create_attendance_file():

    data = """Aarav,Present
Priya,Present
Rahul,Absent
Sneha,Present
Kiran,Absent
Ananya,Present
Vikram,Present
Meera,Absent
Farhan,Present
Divya,Present"""

    with open("/tmp/attendance.txt", "w") as file:
        file.write(data)

    print("Attendance file created successfully")



def read_attendance_file():

    with open("/tmp/attendance.txt", "r") as file:

        lines = file.readlines()

        print("Reading Attendance Records")

        for line in lines:
            print(line.strip())



def count_total_students():

    with open("/tmp/attendance.txt", "r") as file:

        lines = file.readlines()

        total_students = len(lines)

    print(f"Total Students = {total_students}")
```

```
    return total_students


def count_present_students():

    present_count = 0

    with open("/tmp/attendance.txt", "r") as file:

        lines = file.readlines()

        for line in lines:

            name, status = line.strip().split(",")

            if status == "Present":
                present_count += 1

    print(f"Present Students = {present_count}")

    return present_count


def count_absent_students():

    absent_count = 0

    with open("/tmp/attendance.txt", "r") as file:

        lines = file.readlines()

        for line in lines:

            name, status = line.strip().split(",")

            if status == "Absent":
                absent_count += 1

    print(f"Absent Students = {absent_count}")

    return absent_count


def calculate_attendance_percentage():
```

```
    total_students = 10
    present_students = 7

    percentage = (present_students / total_students) * 100

    print(f"Attendance Percentage = {percentage}%")



def list_absent_students():

    print("Absent Students List")

    with open("/tmp/attendance.txt", "r") as file:

        lines = file.readlines()

        for line in lines:

            name, status = line.strip().split(",")

            if status == "Absent":
                print(name)



def generate_attendance_report():

    report = """Daily Attendance Report

Total Students = 10

Present Students = 7

Absent Students = 3

Attendance Percentage = 70%

Status = Needs Improvement"""

    with open("/tmp/attendance_report.txt", "w") as file:
        file.write(report)

    print("Attendance report generated successfully")


with DAG(
```

```
    dag_id="daily_attendance_processor",

    start_date=datetime(2026, 1, 1),

    schedule=None,

    catchup=False,

    tags=["attendance", "student", "practice"]

) as dag:

    task1 = PythonOperator(
        task_id="create_attendance_file",
        python_callable=create_attendance_file
    )

    task2 = PythonOperator(
        task_id="read_attendance_file",
        python_callable=read_attendance_file
    )

    task3 = PythonOperator(
        task_id="count_total_students",
        python_callable=count_total_students
    )

    task4 = PythonOperator(
        task_id="count_present_students",
        python_callable=count_present_students
    )

    task5 = PythonOperator(
        task_id="count_absent_students",
        python_callable=count_absent_students
    )

    task6 = PythonOperator(
        task_id="calculate_attendance_percentage",
        python_callable=calculate_attendance_percentage
    )

    task7 = PythonOperator(
        task_id="list_absent_students",
        python_callable=list_absent_students
    )
```

```python
task8 = PythonOperator(
    task_id="generate_attendance_report",
    python_callable=generate_attendance_report
)

task1 >> task2 >> task3 >> task4 >> task5 >> task6 >> task7 >> task8
```